

Live Preview Architecture

Part 1: Live Preview

Fundamentals

1. What is Live

Preview?

Traditional Preview

vs Live Preview

Traditional Preview

Live Preview

Core Value

Proposition

2. Core Components & Architecture

System

Components

Component

Relationships

3. Basic Data Flow & Communication

Session

Initialization

Entry Change Event

Flow

Part 2: Implementation

Patterns by Architecture

4. Client-Side

Rendering (CSR)

Fundamentals

Initial Load Flow for

CSR:

Entry Change Event

Flow in CSR

4.1 CSR with Live

Preview SDK

(Recommended

Approach)

Generic

Implementation

Pattern

4.2 CSR without

Live Preview SDK

(Manual

Implementation)

Breaking Down the

Communication

Protocol

5. Server-Side

Rendering (SSR)

Fundamentals

Initial Load Flow for

SSR:

Entry Change Event

Flow in SSR

5.1 SSR Workflow

with Live Preview

5.2 Extracting Live

Preview Hash

5.3 Using Hash with

Delivery SDK

5.4 Manual API

Calls with Preview

Hash

5.6 Configure

without Live

Preview SDK:

5.7 Key SSR Points

6. Middleware-Based Architecture Patterns

6.1 Database-

Backed

Architecture Challenge
6.2 The Core Problem
6.3 Solution Approaches
8. Edit Tags: DOM-to-Content Mapping
8.1 What Are Edit Tags?
8.2 How Edit Tags Work
8.3 Generate Edit Tags for Previewed Entry Content
8.4 Applying Edit Tags to DOM
8.5 Special Cases
8.6 Edit Button Configuration
8.7 Key Points for Architects
Part 3: Advanced Integration Concepts
10. Visual Builder: Advanced Live Preview Interface
10.1 What is Visual Builder?
10.3 Prerequisites for Visual Builder
10.4 Enabling Visual Builder
10.5 Empty Placeholder for Multiple Fields
10.6 Troubleshooting Common Issues
10.7 Key Takeaways for Architects
11. Framework-Specific Implementation Examples
11.1 Client-Side Rendering (CSR) Examples
11.2 Server-Side Rendering (SSR) Examples
11.3 Static Site Generation (SSG) Examples
11.4 Advanced Integration Examples

Part 1: Live Preview Fundamentals

1. What is Live Preview?

Live Preview is a real-time content preview system that allows content editors to see changes instantly as they modify content in the CMS, without needing to publish or navigate away from the editing interface.

Traditional Preview vs Live Preview

Traditional Preview	Live Preview
Save → Publish → View in new tab	Edit → See changes instantly
Context switching between CMS and website	Unified editing experience

Time-consuming feedback loop	Immediate visual feedback
Risk of publishing unfinished content	Safe preview environment

Core Value Proposition

- **Instant Feedback:** Content changes appear immediately in the preview
- **Enhanced Productivity:** Editors can iterate quickly without context switching
- **Reduced Errors:** Visual validation before publishing
- **Improved Collaboration:** Stakeholders can review changes in real-time

2. Core Components & Architecture

Live Preview operates through a coordinated system of components:

System Components

1. Contentstack CMS (Entry Editor)

- Where content modifications occur
- Generates entry change events
- Manages Live Preview sessions

2. Live Preview SDK

- Facilitates communication between CMS and the website
- Handles event listening and propagation
- Initialized on the User's website.

3. Preview API vs Delivery API

- Preview API:** Serves unpublished/draft content for preview
- Delivery API:** Serves published content for production
- SDK automatically switches between APIs based on preview mode
- The Preview API supports all routes of the Delivery API and adheres to the same request structure.

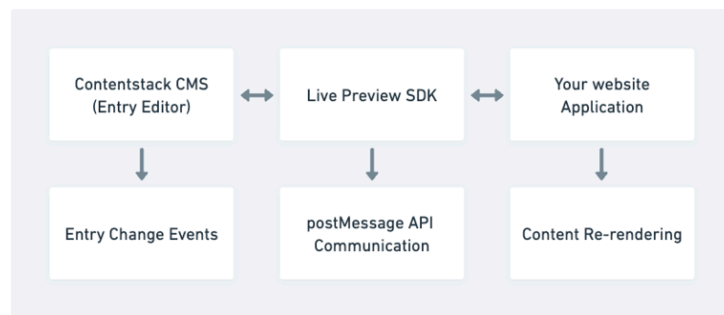
4. Live Preview Hash

- Temporary session identifier
- Enables secure access to preview content
- Scoped to a specific user session

5. Website/Application Layer

- Renders content from Contentstack
- Integrates Live Preview SDK
- Handles content updates and re-rendering

Component Relationships

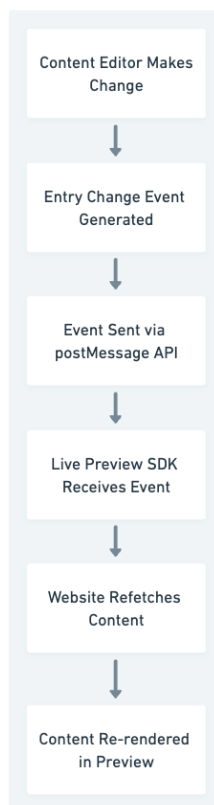


3. Basic Data Flow & Communication

Session Initialization

1. **Session Creation:** When an editor opens an entry, Contentstack generates a unique **Live Preview Hash**
2. **Website Loading:** The website loads inside an iframe within the Live Preview panel

- ### Entry Change Event Flow



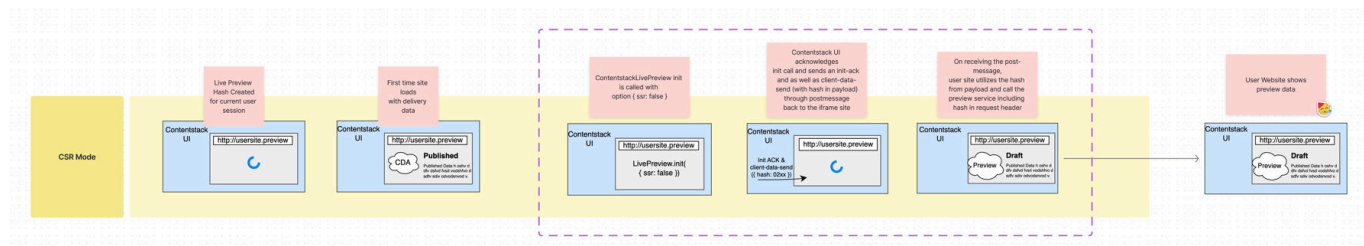
Part 2: Implementation Patterns by Architecture

4. Client-Side Rendering (CSR) Fundamentals

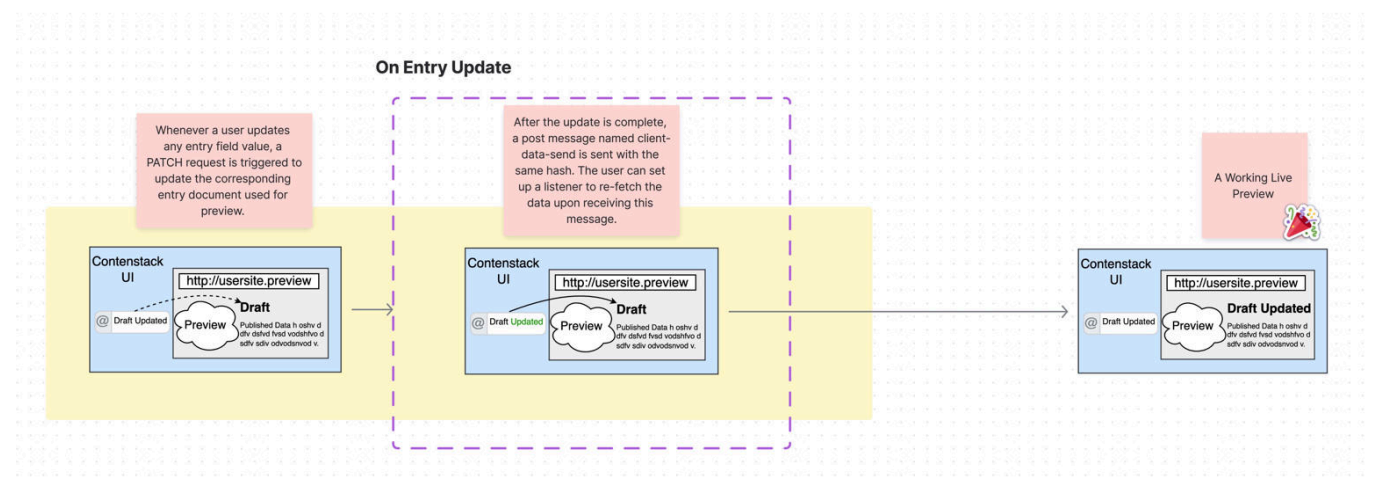
In CSR applications, content is fetched and rendered dynamically in the browser using JavaScript. This architecture provides the most seamless Live Preview experience.

Initial Load Flow for CSR:

-  The Live Preview Hash is generated when a user accesses the entry editor with live preview enabled in their stack. Furthermore, this live hash is refreshed after 30 minutes of inactivity.



Entry Change Event Flow in CSR



4.1 CSR with Live Preview SDK (Recommended Approach)

Generic Implementation Pattern

```
// Initialize Live Preview SDK
import ContentstackLivePreview from '@contentstack/live-preview-utils';

const appURL = new URL("https://app.contentstack.com");

ContentstackLivePreview.init({
  enable: true,
  mode: "preview",
  stackSdk: Stack, // Your Contentstack SDK instance
  clientUrlParams: {
    host: appURL.hostname,
    port: appURL.port,
    protocol: appURL.protocol.split(":")[0] as "http" | "https",
  },
  stackDetails: {
    apiKey: config.API_KEY,
  },
  ssr: false,
});
```

snappify.com

```
// Set up entry change listener
ContentstackLivePreview.onEntryChange() => {
  // 1. Refetch content from Preview API
  const updatedContent = await fetchContentFromPreviewAPI();

  // 2. Architect's Decision Point: Where to store the data
  // Options:
  // - Update component state
  // - Update global state store
  // - Trigger context provider update
  // - Update cache/data layer

  // 3. Redistribute to components
  updateApplicationState(updatedContent);
};
```

All types of CSR applications can be categorized within these frameworks, and the configuration mentioned above will adequately support basic Live Preview functionality.

4.2 CSR without Live Preview SDK (Manual Implementation)

⚠️ We highly recommend against this approach.

When implementing Live Preview without the SDK, architects need to handle the communication and content fetching manually. This approach provides more control but requires additional implementation effort.

Here's a practical example of a custom Live Preview implementation that handles the complete communication flow:

```
1 // Custom Live Preview initialization
2 export function initLivePreview() {
3   // Setup event listeners to receive inputs from the parent window
4   (Contentstack CMS)
5   window.addEventListener("message", (event) => {
6     // Handle different message types from CMS
7     if (event.data.data.type === "reload") {
8       window.location.reload();
9     }
10    if (event.data.data.type === "forward") {
11      window.history.forward();
12    }
13    if (event.data.data.type === "backward") {
14      window.history.back();
15    }
16    if (event.data.type === "history") {
17      console.log("Received history event", event.data);
18    }
19  });
20  // Send initialization message to CMS
21  window.parent.postMessage(
22    {
23      from: "live-preview",
24      type: "init",
25      data: {
26        config: {
27          shouldReload: false, // CSR mode - no reload needed
28          isSSR: false, // Client-side rendering
29          href: window.location.href,
30        },
31      },
32    },
33    "*" // In production, specify exact origin for security
34  );
35 }
```

```
1 // Content refetching with Live Preview integration
2 const onInitialLoad = () => {
3   // Initial content load
4   getContent();
5 }
6 // Make sure only to add Event listeners when rendered
7 // within Contentstack's iframe, and Live Preview is enabled in
8 // config
9 if(!process.env.LIVE_PREVIEW || !isRenderedWithinLPIFrame) {
10   return;
11 }
12 // Initialize Live Preview
13 initLivePreview();
14 // Listen for Live Preview events
15 window.addEventListener("message", (event) => {
16   // Handle initialization acknowledgment
17   if (event.data.type === "init-ack") {
18     console.log("Live Preview initialized successfully",
19       event.data);
20   }
21   // Handle content update events
22   if (event.data.type === "client-data-send") {
23     console.log("Content update received", event.data);
24   }
25   // Store Live Preview hash globally for API calls
26   // It's essential to keep this hash updated for each event, as
27   // it may change.
28   window.livePreviewHash = event.data.data.hash;
29   // Refetch content with preview data
30   getContent();
31 }
32 }
```

```

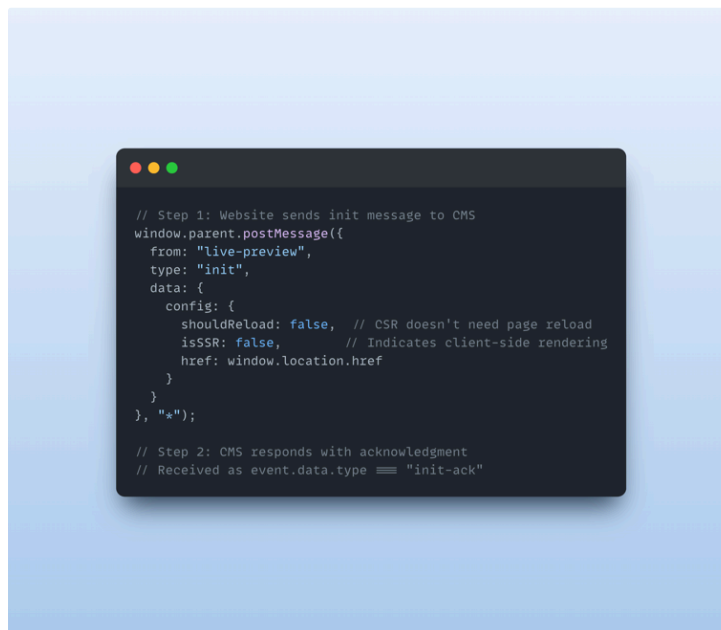
33   });
34
35   // To synchronize URL changes with the navigation on the user
website
36   window.addEventListener("unload", () => {
37     const targetURL = (document.activeElement as
HTMLAnchorElement).href;
38     if (targetURL) {
39       window.parent.postMessage(
40         {
41           from: "live-preview",
42           type: "url-change",
43           data: {
44             targetURL,
45           },
46         },
47         "*"
48       );
49     }
50   });
51 }

```

To effectively register the necessary `postMessage` events, we can utilize lifecycle methods such as `useEffect`, `OnMount`, or `initialize`, depending on the framework in use. Additionally, it is crucial for users to implement `removeEventListener` to ensure that events are properly cleaned up once the component is unmounted or destroyed.

Breaking Down the Communication Protocol

1. Initialization Handshake:



2. Content Update Flow:

```

// Step 1: Editor modifies content in CMS
// Step 2: CMS sends update event
// Received as event.data.type === "client-data-send"

// Step 3: Extract Live Preview hash
window.livePreviewHash = event.data.data.hash;

// Step 4: Refetch content using the hash
getContent(); // This function should use the hash for Preview API calls

```

3. Navigation Control:

```

// CMS can control website navigation
if (event.data.data.type === "reload") {
  window.location.reload();
}

if (event.data.data.type === "forward") {
  window.history.forward();
}

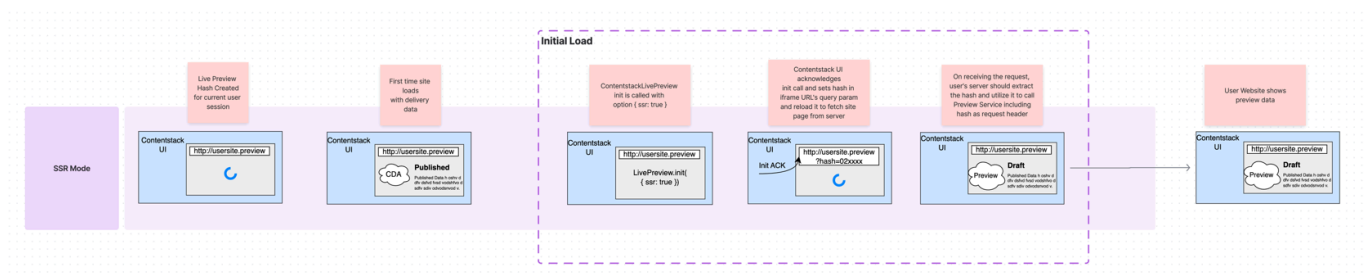
if (event.data.data.type === "backward") {
  window.history.back();
}

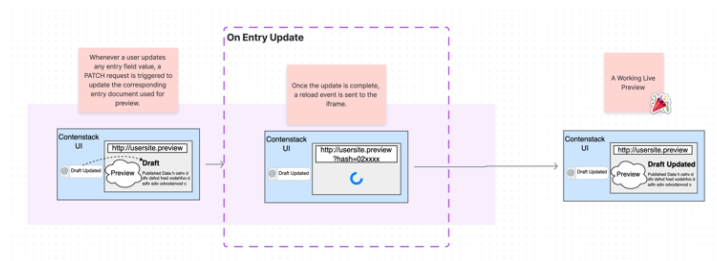
```

5. Server-Side Rendering (SSR) Fundamentals

SSR applications render content on the server before sending HTML to the browser. Live Preview in SSR requires a different approach due to the server-side rendering nature.

Initial Load Flow for SSR:





5.1 SSR Workflow with Live Preview

Live Preview Flow:

1. **Preview Request Initiation:** When a preview request is made, Contentstack appends the Live Preview Hash to the iframe URL's search parameters.
2. **Server-Side Hash Extraction:** The server-side code extracts this query parameter from the URL.
3. **API Configuration:** Use the hash to configure Preview API calls instead of Delivery API.
4. **Server-Side Rendering:** The page is fully rendered server-side with preview content.
5. **Content Updates:** When an entry is updated in Contentstack, the iframe reloads to reflect the latest changes.

5.2 Extracting Live Preview Hash

Generic Hash Extraction:

```

1 // Extract Live Preview hash from URL search parameters
2 function getLivePreviewHash(request) {
3   // From URL object
4   const url = new URL(request.url, `http://${request.headers.host}`);
5   return url.searchParams.get('live_preview');
6
7   // Or from query string directly
8   // const queryString = request.url.split('?')[1];
9   // const params = new URLSearchParams(queryString);
10  // return params.get('live_preview');
11
12  // Or directly set the hash using the Delivery SDK
13  Stack.livePreviewQuery(request.query);
14 }
15
16 // Usage
17 const livePreviewHash = getLivePreviewHash(request);

```

Framework-Specific Examples:

```

1 // Next.js
2 export async function getServerSideProps(context) {
3   const livePreviewHash = context.query.live_preview;
4   // Use hash for API configuration...
5 }
6
7 // Express.js
8 app.get('/page/:slug', (req, res) => {
9   const livePreviewHash = req.query.live_preview;
10  // Use hash for API configuration...
11 });
12
13 // Nuxt.js
14 export default {
15   async asyncData({ query }) {
16     const livePreviewHash = query.live_preview;
17     // Use hash for API configuration...
18   }
19 }

```

5.3 Using Hash with Delivery SDK

If using Contentstack Delivery SDK:

```

1 import contentstack, { QueryOperation, type LivePreviewQuery } from
  "@contentstack/delivery-sdk";
2
3 // Provide queryParams to `livePreviewQuery` and it sets the hash for
  each SDK query.
4 // Also, this updates the SDK to make calls to preview services instead
  of the delivery API.
5 const qs = route.query;

```

```
6 stack.livePreviewQuery(qs as unknown as LivePreviewQuery)
```

Important Note:

Sharing a single instance of the Contentstack SDK across multiple requests (e.g., in a server-side environment like Node.js) with Live Preview enabled can cause unexpected behavior when multiple users make simultaneous requests. When Live Preview is enabled, the SDK may modify its configuration (such as API endpoints) for a specific request. If the same SDK instance handles multiple requests, configuration changes for one user (e.g., User A) can affect others (e.g., User B), leading to incorrect or unexpected content. **To avoid this, create a new Contentstack SDK instance for each request to ensure isolated configurations and prevent interference between requests.**

Example with Proper SDK Instance Management:

```
1 // ❌ Avoid the singleton pattern for SSR
2 const globalStack = contentstack.stack({ /* config */ }); // Don't do this
3
4 // ✅ Create new instance per request or reset configuration
5 function getStackInstance(queryParams) {
6   const stack = contentstack.stack({
7     apiKey: process.env.CONTENTSTACK_API_KEY,
8     deliveryToken: process.env.CONTENTSTACK_DELIVERY_TOKEN,
9     environment: process.env.CONTENTSTACK_ENVIRONMENT
10  });
11
12   // Configure for Live Preview if hash is present
13   if (queryParams.live_preview) {
14     stack.livePreviewQuery(queryParams);
15   }
16
17   return stack;
18 }
19
20 // Usage in SSR
21 export async function getServerSideProps(context) {
22   const stack = getStackInstance(context.query);
23   const entry = await
24     stack.ContentType('page').Entry('entry_uid').fetch();
25
26   return {
27     props: { content: entry }
28   };
29 }
```

5.4 Manual API Calls with Preview Hash

Manual Preview API Configuration:

```
1 // If the delivery SDK is not being used within project then manually
2 // extract LP hash and
3 // provide it as hash for preview api calls
4
5 const getPage = (request) => {
6   const hash = getLivePreviewHash(request);
7   const headers = {
8     'api_key': 'api_key',
9     'access_token': 'delivery_token'
10  };
11   let host = "https://cdn.contentstack.io";
12   if(hash) {
13     headers.live_preview = hash;
14     headers.preview_token = preview_token;
15     host = "https://rest-preview.contentstack.com"
16   }
17
18   // Make api call
19 }
```

5.5 Configure Live Preview SDK in Client

```
1 // Make sure to initialize Live Preview SDK in client side
2
3 ContentstackLivePreview.init({
4   ssr: true,
5   enable: true,
6   mode: "preview",
7   stackDetails: {
8     apiKey: "b1tabcabcabcabc",
9     environment: "env-name"
10  },
11   clientUrlParams: {
12     host: endpoints.application
13   },
14 });
```

```

14     editButton: {
15         enable: true,
16     },
17 };

```

5.6 Configure without Live Preview SDK:

⚠ We highly recommend against this approach. Instead, we recommend utilizing the Live Preview SDK for automatic communication management.

To follow these steps, please refer to the link provided:

[Live Preview Architecture](#) | [4.2 CSR without Live Preview SDK \(Manual Implementation\)](#)

Update init mode to SSR

```

1 window.parent.postMessage(
2   {
3     from: "live-preview",
4     type: "init",
5     data: {
6       config: {
7         shouldReload: true,    // SSR mode - change triggers reload
8         isSSR: true,         // Server-side rendering
9         href: window.location.href,
10      },
11    },
12  },
13  "*" // In production, specify exact origin for security
14 );

```

Additionally, to handle navigation within the user's website within the LP SDK

```

1 export function addLivePreviewQueryTags(link: string): string {
2   try {
3     const docUrl: URL = new URL(document.location.href);
4     const newUrl: URL = new URL(link);
5     const livePreviewHash: string | null =
6       docUrl.searchParams.get("live_preview");
7     const ctUid: string | null =
8       docUrl.searchParams.get("content_type_uid");
9     const entryUid: string | null =
10       docUrl.searchParams.get("entry_uid");
11     if (livePreviewHash && ctUid && entryUid) {
12       newUrl.searchParams.set("live_preview", livePreviewHash);
13       newUrl.searchParams.set("content_type_uid", ctUid);
14       newUrl.searchParams.set("entry_uid", entryUid);
15     }
16     return newUrl.href;
17   } catch (error) {
18     console.error("Error while adding live preview to URL");
19     return link;
20   }
21 }

```

This ensures that the necessary live preview context is retained when navigating within the site.

5.7 Key SSR Points

Essential Steps:

1. **Extract Hash:** `const hash = request.query.live_preview`
2. **Configure API:** Use hash to switch between Preview/Delivery API
3. **Fetch Content:** Server-side fetch with correct API configuration
4. **Render HTML:** Server-side render with preview content
5. **Handle Updates:** Live Preview SDK manages iframe reloads

Important Notes:

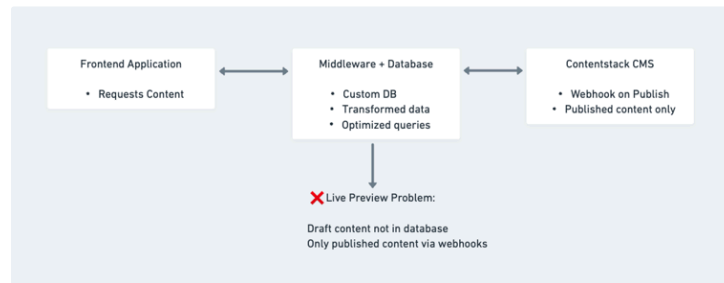
- **Preview API:** Use when `live_preview` hash is present
- **Delivery API:** Use when no hash (published content)
- **No Caching:** Never cache preview content when `live_preview` hash is present
- **Iframe Reloads:** SSR updates require a full page reload (handled by SDK)

This simplified approach focuses on the core mechanics: extracting the hash and using it to configure your API calls correctly.

6. Middleware-Based Architecture Patterns

Middleware-based architectures often involve custom application layers that sit between the frontend and Contentstack, with additional databases maintained through webhook events. These patterns present unique challenges for Live Preview implementation.

6.1 Database-Backed Architecture Challenge



6.2 The Core Problem

Traditional Webhook-Based Flow:

1. Content is published in Contentstack
2. Webhook triggers on the publish event
3. Middleware processes and transforms content
4. Content is stored in a custom database
5. Frontend queries the custom database

Live Preview Challenge:

- **Draft content** is NOT in your custom database
- **Only published content** triggers webhooks
- **Live Preview hash** is only valid for draft content.
- **Database queries** return stale/missing data for preview

It is important to note that this type of architecture can only be supported after implementing additional query capabilities for preview services.

6.3 Solution Approaches

```
1 // Middleware with conditional Live Preview fallback
2 class DatabaseMiddleware {
3   constructor(config) {
4     this.config = config;
5     this.database = config.database;
6     this.contentstackAPI = config.contentstackAPI;
7   }
8
9   async getContent(contentType, entryUid, previewHash) {
10    if (previewHash) {
11      // Live Preview mode - bypass database, query Contentstack
12      // directly
13      console.log('Live Preview detected - fetching from Contentstack API');
14      return await this.fetchFromContentstackAPI(contentType, entryUid, previewHash);
15    }
16
17    // Normal mode - query database
18    return await this.fetchFromDatabase(contentType, entryUid);
19  }
20
21  async fetchFromDatabase(contentType, entryUid) {
22    try {
23      // Query your custom database
24      const result = await this.database.query(
25        'SELECT * FROM content WHERE content_type = ? AND entry_uid = ?',
26        [contentType, entryUid]
27      );
28      return result.rows[0];
29    } catch (error) {
30      console.error('Database query error:', error);
31      throw error;
32    }
33  }
34 }
```

```

34   }
35
36   async fetchFromContentstackAPI(contentType, entryUid, previewHash) {
37     try {
38       // Configure for Preview API
39       const apiConfig = {
40         baseUrl: 'https://rest-preview.contentstack.com',
41         headers: {
42           'api_key': this.config.contentstackAPI.apiKey,
43           'preview_token': this.config.contentstackAPI.previewToken,
44           'live_preview': previewHash,
45           'Content-Type': 'application/json'
46         },
47         params: {}
48       };
49
50       // Build API URL
51       const url =
52         `${apiConfig.baseUrl}/v3/content_types/${contentType}/entries/${entryU
53         id}`;
54       const params = new URLSearchParams(apiConfig.params);
55
56       const response = await fetch(`${url}?${params.toString()}`, {
57         headers: apiConfig.headers
58       });
59
60       if (!response.ok) {
61         throw new Error(`Preview API error: ${response.status}`);
62       }
63
64       const data = await response.json();
65
66       // Transform API response to match database structure
67       return this.transformContentstackData(data.entry);
68     } catch (error) {
69       console.error('Contentstack API fallback error:', error);
70       throw error;
71     }
72   }
73
74   transformContentstackData(entry) {
75     // Transform Contentstack API response to match your database
76     schema
77     // This is crucial for maintaining consistency
78     return {
79       id: entry.uid,
80       title: entry.title,
81       content: entry.content,
82       // Apply same transformations as webhook processing
83       processed_data: this.processContent(entry),
84       created_at: entry.created_at,
85       updated_at: entry.updated_at
86     };
87   }
88
89   processContent(entry) {
90     // Apply the same processing logic used in webhook handlers
91     // This ensures preview content matches published content
92     structure
93     return {
94       // Your custom processing logic here
95       word_count: entry.content?.split(' ').length || 0,
96       reading_time: Math.ceil((entry.content?.split(' ').length || 0)
97       / 200),
98       // Other transformations...
99     };
100   }
101 }

```

You can also opt to retrieve preview data and save it in a separate collection, utilizing a hash and TTL. This collection can then be used to resolve data for the Preview.

8. Edit Tags: DOM-to-Content Mapping

Edit tags are data attributes that link DOM elements to specific content fields in Contentstack, enabling Visual Builder functionality and contextual editing.

8.1 What Are Edit Tags?

Edit tags create a direct mapping between your rendered content and the original content structure in Contentstack. They enable:

- **Visual Builder Integration:** On-page editing capabilities
- **Contextual Editing:** Click directly on content to edit it
- **Field Navigation:** Quick access to specific fields in CMS
- **Content Field Identification:** Link DOM elements to content fields

8.2 How Edit Tags Work

Basic Concept:

```
1 <!-- Edit tag links this DOM element to a specific content field -->
2 <h1 data-csdp="page.bl123456.en-us.title">Welcome to Our Website</h1>
```

Edit Tag Structure:

```
1 data-csdp="content_type.entry_uid.locale.field_path"
```

8.3 Generate Edit Tags for Previewed Entry Content

After retrieving data using the Contentstack [JavaScript Delivery SDK](#), pass the entry to the `addEditableTags()` function to add edit tags to the previewed entry content:

```
1 addEditableTags(entry, content_type_uid, tagsAsObject, locale)
```

By default, `tagsAsObject` is false, adding `data-csdp` as a string like this:

```
1 title: 'data-csdp=path.to.field'
```

When `tagsAsObject` is set to `true`, the `data-csdp` attribute is returned in object format as shown below:

```
1 { 'data-csdp': 'path.to.field' }
```

The following sample shows how the code looks after applying the `addEditableTags()` method:

```
1 let entry = await Stack.ContentType("content_type_uid").Query()
2   .where("url", URL)
3   .find();
4 addEditableTags(entry[0][0], "content_type_uid", false, 'en-us')
```

8.4 Applying Edit Tags to DOM

React Example:

```
1 // Use edit tags from API response
2 <h1 {...entry.$.title}>{entry.title}</h1>
3 <p {...entry.$.description}>{entry.description}</p>
```

8.5 Special Cases

Multiple Fields (Arrays/Modular Blocks):

```
1 // For modular blocks, use double underscore pattern
2 <div {...entry.$?.page_components}>
3   {entry.page_components?.map((component, index) => (
4     <div key={index} {...entry.$?.['page_components__${index}']}>
5       <Component data={component} />
6     </div>
7   )}
8 </div>
```

Nested Fields:

```
1 // For nested objects
2 <div {...entry.hero.$.title}>{entry.hero.title}</div>
3 <div {...entry.hero.$.description}>{entry.hero.description}</div>
```

8.6 Edit Button Configuration

```
1 // Configure edit button behavior
2 ContentstackLivePreview.init({
3   // ... other config
4   editButton: {
5     enable: true,
6     position: 'top-right', // 8 predefined positions available
7     exclude: ["outsideLivePreviewPortal"] // Optional exclusions
8   }
9 });
```

8.7 Key Points for Architects

Why Edit Tags Are Significant:

1. **Visual Builder Foundation:** Edit tags are required for Visual Builder to work
2. **Content-DOM Mapping:** They create the link between CMS fields and rendered content
3. **Contextual Editing:** Enable editors to click directly on content to edit it
4. **Field-Level Precision:** Allow editing of specific fields without navigating through the CMS

Implementation Considerations:

- Edit tags are automatically generated by the `addEditableTags` function
- They must be applied to DOM elements during rendering
- Different patterns exist for simple fields, nested fields, and arrays
- React applications should use object destructuring for a cleaner implementation

Performance Impact:

- Edit tags are lightweight data attributes
- They don't affect page performance significantly
- Visibility can be managed through flags and configured according to specific conditions.

Debugging edit tags:

```
1 function createEditTagDebugger() {
2   let isActive = false;
3   const tooltips = [];
4   let activeHighlight = null;
5
6   return {
7     toggle() {
8       if (isActive) {
9         this.hide();
10      } else {
11        this.show();
12      }
13    },
14
15    show() {
16      if (isActive) return;
17
18      const editTagElements = document.querySelectorAll('[data-
19        cslp]');
20
21      editTagElements.forEach(element => {
22        // Add outline
23        element.style.outline = '2px dashed #007bff';
24        element.style.position = element.style.position ||
25        'relative';
26
27        // Create tooltip
28        const tooltip = document.createElement('div');
29        const editTagValue = element.getAttribute('data-cslp');
30
31        tooltip.textContent = editTagValue;
32        tooltip.className = 'cslp-debug-tooltip';
33        tooltip.style.cssText = `
34          position: absolute;
35          top: -35px;
36          left: 0;
37          background: #007bff;
38          color: white;
39          padding: 4px 8px;
40          font-size: 11px;
41          border-radius: 4px;
42          z-index: 1000;
43          white-space: nowrap;
44          pointer-events: auto;
45          max-width: 280px;
46          overflow: hidden;
47          text-overflow: ellipsis;
48          box-shadow: 0 2px 8px rgba(0,0,0,0.2);
49          font-family: monospace;
50          cursor: pointer;
51          transition: all 0.2s ease;
52        `;
53
54        // Add click handler to highlight element
55        tooltip.addEventListener('click', (e) => {
56          e.stopPropagation();
57          this.highlightElement(element);
58        });
59
60        // Add hover handlers for collapsed tooltips
61        let hoverTooltip = null;
62        let hoverTimeout = null;
63
64        tooltip.addEventListener('mouseenter', (e) => {
65          // Only show hover tooltip if text is truncated
66          if (tooltip.scrollHeight > tooltip.clientHeight ||
67            editTagValue.length > 40) {
68            hoverTimeout = setTimeout(() => {
69              hoverTooltip = this.createHoverTooltip(editTagValue,
70                e);
71            }, 500); // 500ms delay
72          }
73        });
74      });
75    },
76
77    hide() {
78      // Remove outline
79      editTagElements.forEach(element => {
80        element.style.outline = '';
81      });
82
83      // Hide tooltip
84      if (activeHighlight) {
85        activeHighlight.style.outline = '';
86        activeHighlight.style.position = '';
87      }
88
89      // Hide hover tooltip
90      if (hoverTooltip) {
91        hoverTooltip.remove();
92        hoverTimeout = null;
93      }
94
95      activeHighlight = null;
96      isActive = false;
97    },
98
99    highlightElement(element) {
100      element.style.outline = '2px solid #007bff';
101      activeHighlight = element;
102      isActive = true;
103    },
104
105    createHoverTooltip(editTagValue, e) {
106      const hoverTooltip = document.createElement('div');
107      hoverTooltip.textContent = editTagValue;
108      hoverTooltip.className = 'cslp-debug-hover-tooltip';
109      hoverTooltip.style.cssText = `
110        position: absolute;
111        top: 0;
112        left: 0;
113        background: #007bff;
114        color: white;
115        padding: 4px 8px;
116        font-size: 11px;
117        border-radius: 4px;
118        z-index: 1000;
119        white-space: nowrap;
120        pointer-events: auto;
121        max-width: 280px;
122        overflow: hidden;
123        text-overflow: ellipsis;
124        box-shadow: 0 2px 8px rgba(0,0,0,0.2);
125        font-family: monospace;
126        cursor: pointer;
127        transition: all 0.2s ease;
128      `;
129
130      // Add click handler to highlight element
131      hoverTooltip.addEventListener('click', (e) => {
132        e.stopPropagation();
133        this.highlightElement(element);
134      });
135
136      // Add hover handlers for collapsed tooltips
137      let hoverTimeout = null;
138
139      hoverTooltip.addEventListener('mouseenter', (e) => {
140        // Only show hover tooltip if text is truncated
141        if (hoverTooltip.scrollHeight > hoverTooltip.clientHeight ||
142          editTagValue.length > 40) {
143          hoverTimeout = setTimeout(() => {
144            hoverTooltip = this.createHoverTooltip(editTagValue,
145              e);
146          }, 500); // 500ms delay
147        }
148      });
149
150      // Add mouseleave handler to hide tooltip
151      hoverTooltip.addEventListener('mouseleave', (e) => {
152        hoverTimeout = null;
153      });
154
155      // Append hover tooltip to document
156      document.body.appendChild(hoverTooltip);
157
158      return hoverTooltip;
159    },
160  };
161 }
```

```

69     });
70
71     tooltip.addEventListener('mouseleave', () => {
72         if (hoverTimeout) {
73             clearTimeout(hoverTimeout);
74             hoverTimeout = null;
75         }
76         if (hoverTooltip) {
77             hoverTooltip.remove();
78             hoverTooltip = null;
79         }
80     });
81
82     tooltip.addEventListener('mousemove', (e) => {
83         if (hoverTooltip) {
84             // Update hover tooltip position
85             hoverTooltip.style.left = `${e.pageX + 10}px`;
86             hoverTooltip.style.top = `${e.pageY - 10}px`;
87         }
88     });
89
90     element.appendChild(tooltip);
91     tooltips.push({ element, tooltip });
92
93     // Handle overflow after DOM insertion
94     requestAnimationFrame(() => {
95         this.adjustTooltipPosition(tooltip);
96     });
97 });
98
99 isActive = true;
100 console.log(`🔥 Highlighted ${editTagElements.length} edit
tags`);
101 },
102
103 hide() {
104     if (!isActive) return;
105
106     tooltips.forEach(({ element, tooltip }) => {
107         // Remove outline
108         element.style.outline = '';
109
110         // Remove tooltip
111         if (tooltip.parentNode) {
112             tooltip.parentNode.removeChild(tooltip);
113         }
114     });
115
116     tooltips.length = 0;
117     isActive = false;
118
119     // Clear active highlight
120     if (activeHighlight) {
121         this.clearHighlight();
122     }
123
124     console.log(`🔥 Edit tag highlighting disabled`);
125 },
126
127 createHoverTooltip(fullText, mouseEvent) {
128     const hoverTooltip = document.createElement('div');
129     hoverTooltip.textContent = fullText;
130     hoverTooltip.className = 'cslp-hover-tooltip';
131     hoverTooltip.style.cssText = `
132     position: absolute;
133     left: ${mouseEvent.pageX + 10}px;
134     top: ${mouseEvent.pageY - 10}px;
135     background: #333;
136     color: white;
137     padding: 6px 10px;
138     font-size: 12px;
139     border-radius: 4px;
140     z-index: 10001;
141     white-space: nowrap;
142     pointer-events: none;
143     box-shadow: 0 4px 12px rgba(0,0,0,0.3);
144     font-family: monospace;
145     max-width: 400px;
146     word-break: break-all;
147     white-space: normal;
148     line-height: 1.3;
149 `;
150
151     document.body.appendChild(hoverTooltip);
152
153     // Adjust position if it goes outside viewport
154     const rect = hoverTooltip.getBoundingClientRect();
155     if (rect.right > window.innerWidth - 10) {
156         hoverTooltip.style.left = `${mouseEvent.pageX - rect.width -
157 10}px`;
158     }
159     if (rect.bottom > window.innerHeight - 10) {
160         hoverTooltip.style.top = `${mouseEvent.pageY - rect.height -
161 10}px`;
162     }

```



```

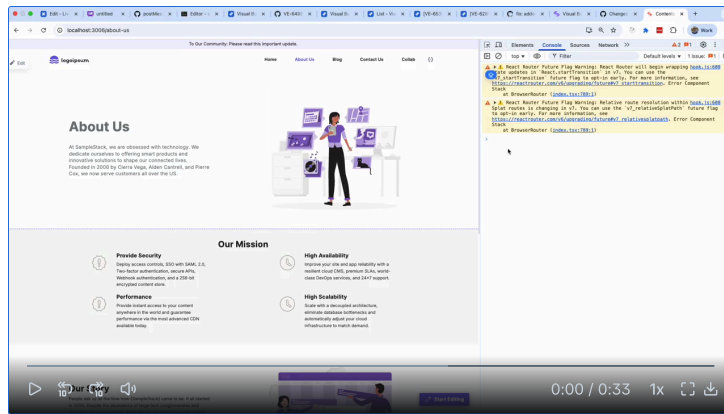
161     return hoverTooltip;
162 },
163
164 highlightElement(element) {
165     // Clear previous highlight
166     this.clearHighlight();
167
168     // Add strong highlight to clicked element
169     const originalOutline = element.style.outline;
170     element.style.outline = '4px solid #ff6b35';
171     element.style.outlineOffset = '2px';
172     element.style.backgroundColor = 'rgba(255, 107, 53, 0.1)';
173     element.style.transition = 'all 0.3s ease';
174
175     // Scroll element into view
176     element.scrollIntoView({
177         behavior: 'smooth',
178         block: 'center',
179         inline: 'center'
180     });
181
182     // Store highlight info
183     activeHighlight = {
184         element,
185         originalOutline,
186         originalBackground: element.style.backgroundColor
187     };
188
189     // Show element info in console
190     const editTag = element.getAttribute('data-cslp');
191     console.log('🔗 Highlighted element:', {
192         editTag,
193         element,
194         tagName: element.tagName,
195         className: element.className,
196         textContent: element.textContent?.substring(0, 100) + '...'
197     });
198
199     // Auto-clear highlight after 5 seconds
200     setTimeout(() => {
201         if (activeHighlight && activeHighlight.element === element) {
202             this.clearHighlight();
203         }
204     }, 5000);
205 },
206
207 clearHighlight() {
208     if (activeHighlight) {
209         const { element, originalOutline, originalBackground } =
210 activeHighlight;
211         element.style.outline = originalOutline;
212         element.style.outlineOffset = '';
213         element.style.backgroundColor = originalBackground;
214         element.style.transition = '';
215         activeHighlight = null;
216     }
217 },
218
219 adjustTooltipPosition(tooltip) {
220     const rect = tooltip.getBoundingClientRect();
221     const viewportWidth = window.innerWidth;
222     const viewportHeight = window.innerHeight;
223     const parentRect =
224 tooltip.parentElement.getBoundingClientRect();
225
226     // Handle horizontal overflow
227     if (rect.right > viewportWidth - 10) {
228         tooltip.style.left = 'auto';
229         tooltip.style.right = '0';
230
231         // If still overflowing, truncate more aggressively
232         if (rect.left < 10) {
233             tooltip.style.maxWidth = `${Math.max(200, viewportWidth -
234 parentRect.left - 20)}px`;
235         }
236     }
237
238     // Handle vertical overflow
239     if (rect.top < 10) {
240         tooltip.style.top = 'auto';
241         tooltip.style.bottom = '-35px';
242
243         // Check if it overflows bottom after repositioning
244         const newRect = tooltip.getBoundingClientRect();
245         if (newRect.bottom > viewportHeight - 10) {
246             // Position to the side if no vertical space
247             tooltip.style.top = '0';
248             tooltip.style.bottom = 'auto';
249             tooltip.style.left = '100%';
250             tooltip.style.right = 'auto';
251             tooltip.style.marginLeft = '10px';
252         }
253     }
254 }

```

```

253 // Handle very long edit tags
254 if (tooltip.textContent.length > 50) {
255     tooltip.style.maxWidth = '250px';
256     tooltip.style.whiteSpace = 'normal';
257     tooltip.style.wordBreak = 'break-all';
258     tooltip.style.maxHeight = '80px';
259     tooltip.style.overflowY = 'auto';
260     tooltip.style.lineHeight = '1.2';
261 }
262 },
263
264 // Get statistics about edit tags
265 getStats() {
266     const editTagElements = document.querySelectorAll('[data-
267     cslp]');
268     const stats = {
269         total: editTagElements.length,
270         byContentType: {},
271         byFieldType: {},
272     };
273     editTagElements.forEach(element => {
274         const editTag = element.getAttribute('data-cslp');
275         const parts = editTag.split('.');
276
277         if (parts.length >= 3) {
278             const contentType = parts[0];
279             stats.byContentType[contentType] =
280             (stats.byContentType[contentType] || 0) + 1;
281         }
282     });
283     return stats;
284 },
285
286 // Find elements by content type
287 findByContentType(contentType) {
288     const elements = document.querySelectorAll(`[data-
289     cslp*="${contentType}."`);
290     console.log(`Found ${elements.length} elements for content
291     type: ${contentType}`);
292     return elements;
293 },
294
295 // Find elements by field path
296 findByFieldPath(fieldPath) {
297     const elements = document.querySelectorAll(`[data-
298     cslp*="${fieldPath}"]`);
299     console.log(`Found ${elements.length} elements containing field
300     path: ${fieldPath}`);
301     return elements;
302 },
303
304 // Usage
305 const editTagDebugger = createEditTagDebugger();
306
307 // Toggle highlighting
308 editTagDebugger.toggle();
309
310 // Get statistics
311 console.table(editTagDebugger.getStats());
312
313 // Keyboard shortcuts
314 document.addEventListener('keydown', (e) => {
315     // Ctrl/Cmd + Shift + E to toggle edit tag highlighting
316     if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.key === 'E') {
317         e.preventDefault();
318         editTagDebugger.toggle();
319     }
320
321     // Escape to clear highlight
322     if (e.key === 'Escape') {
323         editTagDebugger.clearHighlight();
324     }
325 });
326
327 // Click outside to clear highlight
328 document.addEventListener('click', (e) => {
329     if (!e.target.closest('.cslp-debug-tooltip')) {
330         editTagDebugger.clearHighlight();
331     }
332 });
333
334 // Console helpers
335 window.editTagDebugger = editTagDebugger;
336 console.log('🔗 Edit Tag Debugger loaded! Use
337 editTagDebugger.toggle() or Ctrl+Shift+E');

```



For detailed implementation guides, examples, and advanced configurations, refer to the official Contentstack documentation:

[Set Up Live Edit Tags for Entries with REST](#)

Part 3: Advanced Integration Concepts

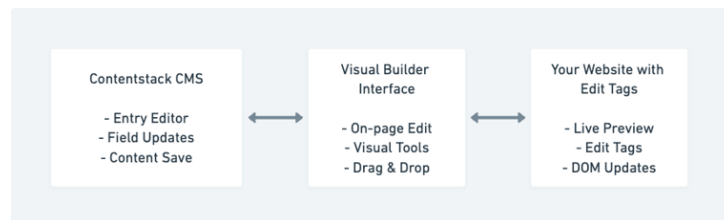
10. Visual Builder: Advanced Live Preview Interface

Visual Builder is Contentstack's advanced on-page editing interface that works on top of Live Preview and edit tags to provide a WYSIWYG editing experience directly on your website.

10.1 What is Visual Builder?

Visual Builder extends Live Preview functionality by providing:

- **On-page editing:** Edit content directly on your website
- **Visual content management:** See changes in real-time as you edit
- **Contextual editing:** Edit content in the context of your actual website
- **Multi-field operations:** Add, delete, and reorder multiple field items



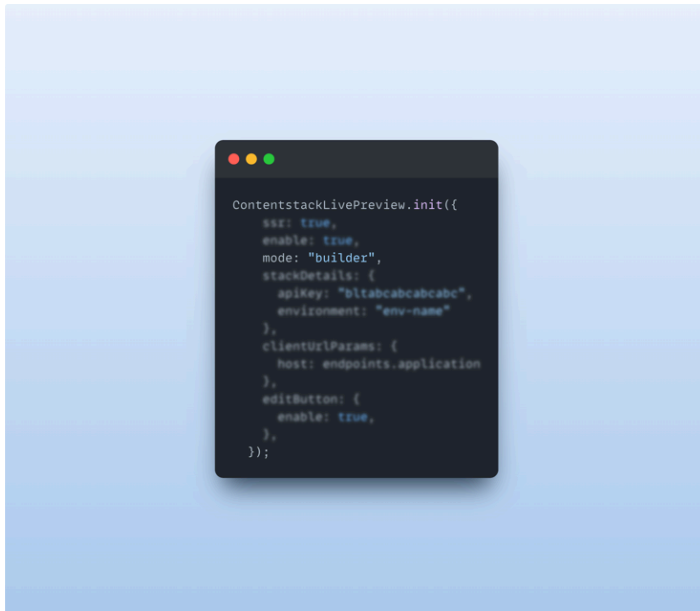
10.3 Prerequisites for Visual Builder

Required Components:

1. **Live Preview Setup:** Visual Builder requires Live Preview to be configured
2. **Edit Tags:** All editable content must have proper edit tags

10.4 Enabling Visual Builder

1. Upgrade Live Preview SDK to the latest version (3.0 and above)
2. Configure mode to "builder" in init.



10.5 Empty Placeholder for Multiple Fields

For better UX, when modular blocks fields are not populated:

```
1 import { VB_EmptyBlockParentClass } from '@contentstack/live-preview-
  utils'
2
3 <div
4   {...(entry.$?.page_components || {})}
5   className={pageComponents?.length ? undefined :
     VB_EmptyBlockParentClass}
6 >
7   {/* Block components */}
8 </div>
```

10.6 Troubleshooting Common Issues

CORS Errors:

- Ensure your website is iframe-compatible
- Check Content Security Policy settings

Edit Tags Not Working:

- Verify SDK versions meet minimum requirements
- Check the edit tag syntax and structure
- Ensure Live Preview is properly configured first

10.7 Key Takeaways for Architects

1. **Foundation Dependency:** Visual Builder requires Live Preview as a prerequisite
2. **SDK Requirements:** Specific minimum versions are required for functionality
3. **Edit Tag Strategy:** Proper edit tag implementation is crucial for Visual Builder success

For detailed implementation instructions and advanced configuration options, refer to the official Contentstack documentation: [Set Up Visual Builder for Your Website](#)

11. Framework-Specific Implementation Examples

This section provides categorized implementation examples for different rendering strategies and frameworks, enabling architects to choose the most appropriate approach for their specific use case.

Note: The examples provided below are not officially available for customer use. Please consult with the team before sharing this information with customers.

11.1 Client-Side Rendering (CSR) Examples

React with Vite

- **Repository:** <https://github.com/timbenniks/contentstack-implementation-guide-react> [Connect your Github account](#)
- **Use Case:** Single Page Applications (SPAs) with client-side rendering
- **Key Features:**
 - Basic SDK initialization
 - Live Preview setup with Vite development server
 - Client-side content fetching

Angular Implementation

- **Repository:** <https://github.com/timbenniks/contentstack-implementation-guide-angular>
- **Use Case:** Enterprise Angular applications with client-side rendering
- **Key Features:**
 - TypeScript implementation
 - Component-based architecture

11.2 Server-Side Rendering (SSR) Examples

Next.js SSR

- **Repository:** [contentstack-implementation-guide-next-ssr](#)
- **Use Case:** Server-side rendered applications with dynamic content
- **Key Features:**
 - Server-side content fetching
 - Dynamic routing with Live Preview
 - Optimized for SEO and performance

Nuxt.js SSR

- **Repository:** [contentstack-implementation-guide-nuxt-ssr](#)
- **Use Case:** Vue.js applications with server-side rendering
- **Key Features:**
 - Vue.js ecosystem integration
 - Server-side content fetching
 - Nuxt.js-specific optimizations

11.3 Static Site Generation (SSG) Examples

Next.js SSG

- **Repository:** [contentstack-implementation-guide-next-ssg](#)
- **Use Case:** Static sites with build-time content generation

Next.js SSG with GraphQL

- **Repository:** [contentstack-next-ssg-graphql](#)
- **Use Case:** Static sites using GraphQL for content queries

11.4 Advanced Integration Examples

Next.js with Middleware

- **Repository:** [contentstack-implementation-guide-next-middleware](#)

Next.js with Personalization and Visual Builder

- **Repository:** [contentstack-next-middleware-personalize-visual-builder](#)
- **Use Case:** Personalized content with Visual Builder support

Next.js with GraphQL

- **Repository:** [contentstack-implementation-guide-next-graphql](#)
- **Use Case:** GraphQL-first Next.js applications

Standard Next.js Implementation

- **Repository:** [contentstack-implementation-guide-next](#)
- **Use Case:** Basic Next.js integration with Contentstack

Nuxt.js Standard Implementation

- **Repository:** [contentstack-implementation-guide-nuxt](#)
- **Use Case:** Vue.js applications with Nuxt.js framework

Rendering Strategy	Framework	Repository	Use Case	SEO	Complexity
CSR	React	react	SPAs, Interactive Apps	Poor	Low
CSR	Angular	angular	Enterprise Apps	Poor	Medium
SSR	Next.js	next-ssr	Dynamic Content	Excellent	Medium
SSR	Nuxt.js	nuxt-ssr	Vue.js Dynamic	Excellent	Medium
SSG	Next.js	next-ssg	Static Sites	Excellent	Low
SSG	Next.js + GraphQL	next-ssg-graphql	Complex Static	Excellent	Medium
Hybrid	Next.js + Middleware	next-middleware	Edge Processing	Excellent	High
Advanced	Next.js + Personalize	next-personalize-vb	Personalization	Excellent	High